C.E.S.A.R school

Aprendizado de Máquina

Estratégias para treino e teste

2026 | Março

Um modelo que funciona bem no treino nem sempre funciona bem na prática

Erro de treino ≠ erro real

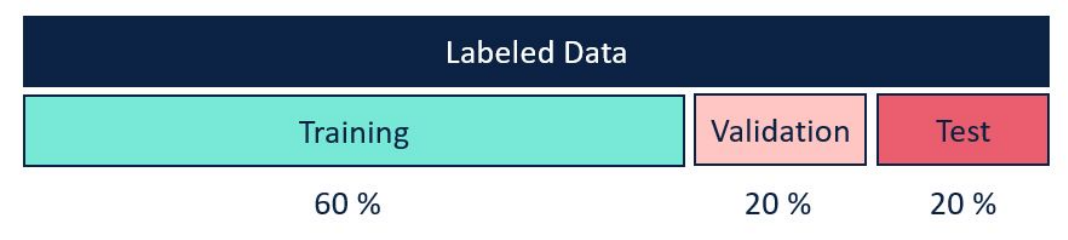
Até agora...

- **Dividimos o conjunto de dados em dois subconjuntos**
  - **Treino**: usado para treinar o modelo de aprendizado de máquina
  - **Teste**: usado para avaliar o modelo finalmente treinado
- **Problema**: uma única divisão pode ser enganosa.

Treino	Teste

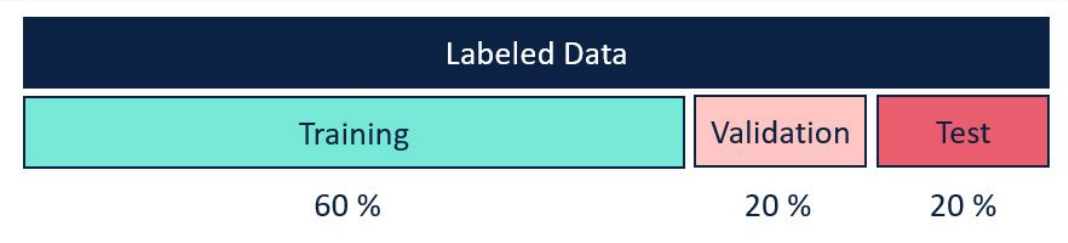
Hold-out

- **Hold-out**: método comumente utilizado que divide o conjunto de dados aleatoriamente em **treino**, **validação** e **testes**
  - **Treino**: usado para treinar o modelo de aprendizado de máquina
  - **Validação**: usado para auxiliar o processo de fine-tuning do modelo
    - Ajuste de hiperparâmetros
    - Prevenção de overfitting
  - **Teste**: usado para avaliar o modelo finalmente treinado



Hold-out

- **Mais dados de treino** tem melhor aprendizado
- **Mais dados de teste** tem avaliação mais confiável
- **Trade-off entre aprendizado e avaliação**



Hold-out: Dividindo a base de dados

1. Dividimos os dados em um conjunto de treino (**X_train, y_train**)
2. 30% dos dados foram alocados num conjunto temporário (**X_temp, y_temp**)
3. O conjunto temporário resulta então em um **conjunto de validação e teste**, contendo 50% dos dados (dos 30% separados do dataset original)

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
random_state=42)
```

Hold-out: problemas

- **Alta variabilidade nos resultados**
  - O desempenho do modelo muda muito dependendo de como você divide os dados
- **Subutilização do conjunto de treino**
  - **Os dados destinados a teste podem ter potencial de aprendizado**
- Dependendo do tamanho do dataset **pode ser inviável separar os dados em 3 subconjuntos**
- **E quando os dados são desbalanceados?**
  - Uma estratégia de estratificação pode ajudar

Hold-out: estratificado

- Podemos buscar por uma divisão proporcional das classes entre os conjuntos de treino, teste e validação

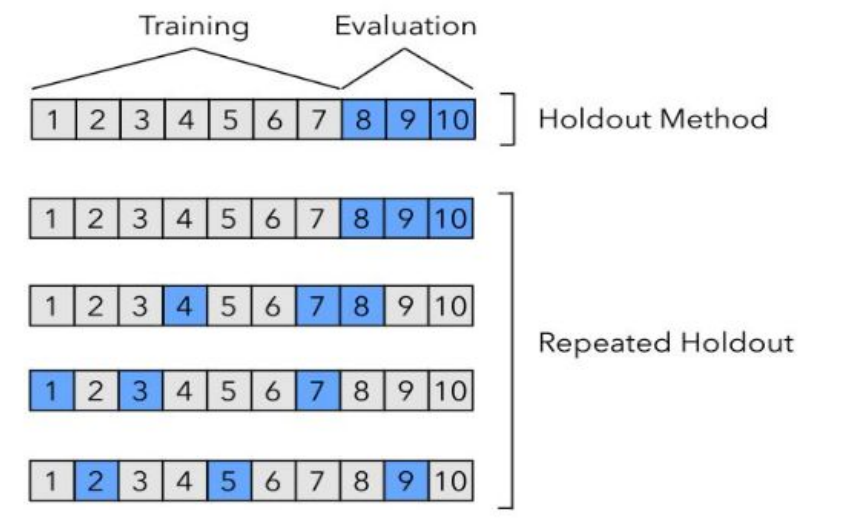
```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, stratify = y,
random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, stratify = y_temp,
test_size=0.5, random_state=42)
```

Hold-out: resumo

- Apesar de simples, apresenta problemas importantes
  - **Alta variabilidade**: o desempenho muda muito a cada divisão
  - **Dependência da aleatoriedade**: o resultado depende de como os dados foram separados
  - **Uso ineficiente dos dados**: parte do dataset não é utilizada no treinamento

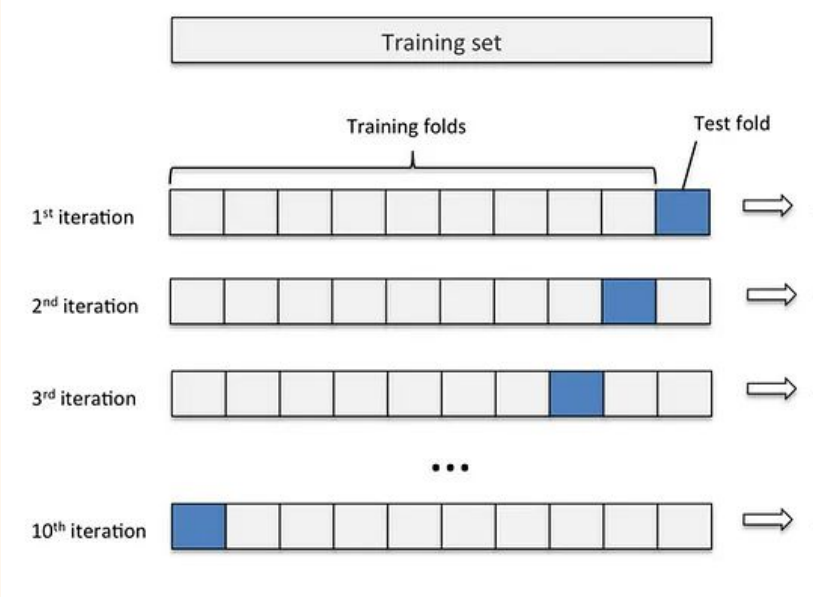
Repeated Hold-out

- Executar múltiplas divisões aleatórias
- Resultado final é representado pela média das métricas
  - Reduz a variabilidade



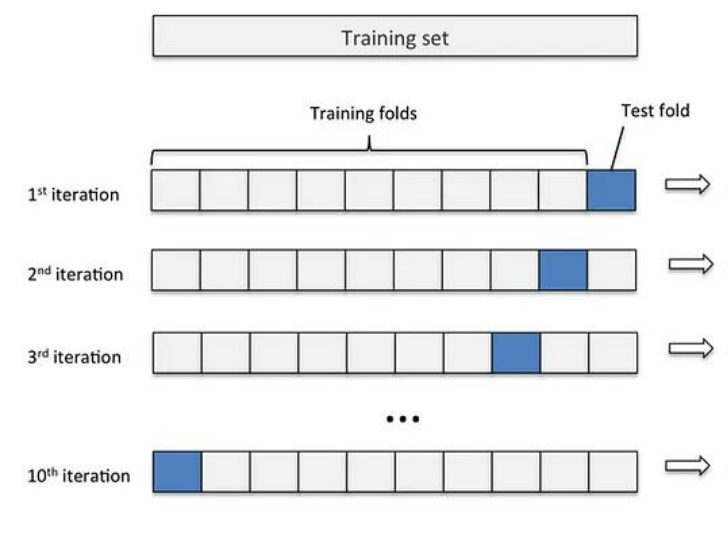
Validação cruzada

- **Dividir aleatoriamente o dataset em k grupos igualmente** para múltiplas rodadas de treino e validação



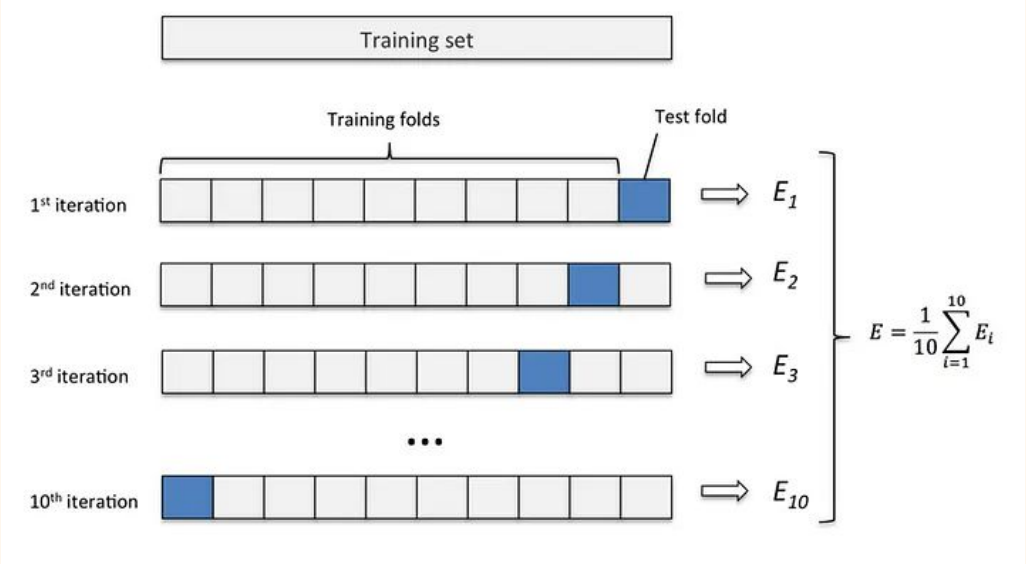
Validação cruzada

- Um grupo é utilizado como validação e os demais grupos são usados para treino em **k iterações**
  - Cada combinação de subconjuntos será usada pelo menos uma vez



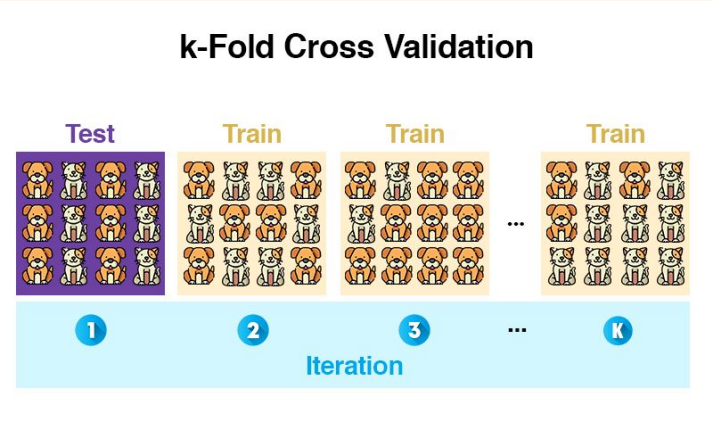
Validação cruzada

- Para cada iteração, fazemos avaliações de performance do modelo
  - Ex.: **computar a acurácia alcançada pelo modelo em cada iteração e obter uma média**



Validação cruzada

- **É uma boa maneira de avaliar como o modelo se comporta diante de variações nos dados de treinamento**
- Menos impactado pela divisão dos dados
- Normalmente usa-se k igual a 5 ou a 10
- Trade-off:
  - k pequeno tem maior viés
  - k grande tem maior custo

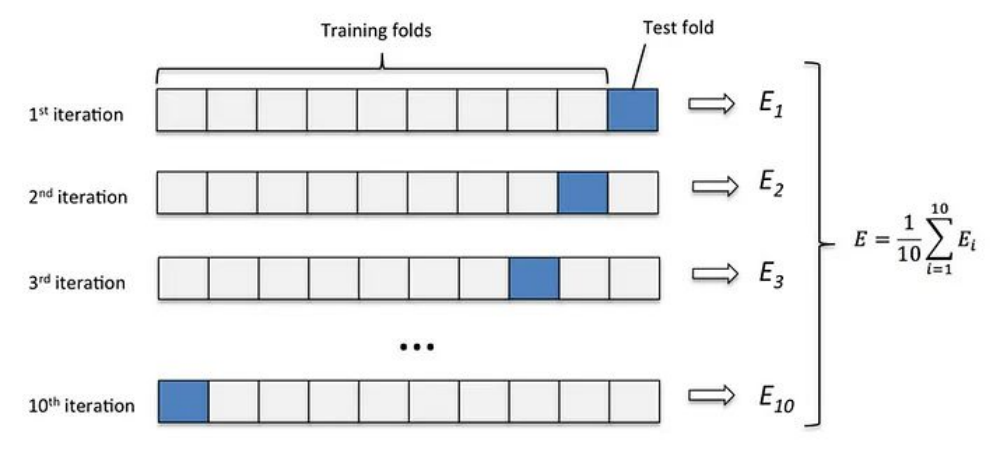


Validação cruzada: implementação

```
from sklearn.model_selection import KFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import numpy as np

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = []

for train_index, val_index in kf.split(X):
    X_train, X_val = X[train_index], X[val_index]
    y_train, y_val = y[train_index], y[val_index]

    # treino do modelo e predição
    # avaliação
    acc = accuracy_score(y_val, y_pred)
    scores.append(acc)

# resultado final
print("Acurácias por fold:", scores)
print("Acurácia média:", np.mean(scores))
print("Desvio padrão:", np.std(scores))
```

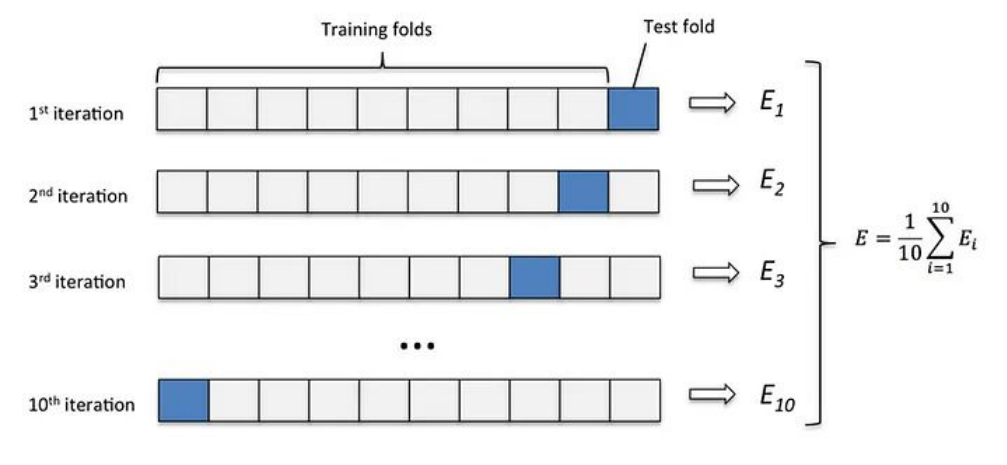
Leave-one-out

- É um caso específico do k-fold cross validation (**caso extremo**)
  - **k é igual ao número total de dados N**



Leave-one-out

- Usa quase todos os dados para treino
- Alto custo computacional e alta variância
- **Ótima aplicação**: datasets muito pequenos



Comparação dos métodos

Método	Viés	Variância	Custo
Hold-out	Médio	Alto	Baixo
K-Fold	Baixo	Médio	Médio
LOO	Baixo	Alto	Alto

Data Leakage

- Erro crítico:
  - Usar informação do teste no treino
  - Desempenho artificialmente alto
- Exemplos:
  - Normalizar antes da divisão
  - Ajustar modelo usando teste
- Correto:

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # aprende só no treino
X_test = scaler.transform(X_test)        # aplica no teste
```

Questões

- Se o hold-out deu 95% e o K-Fold deu 88%, qual você acredita?
- Se o K-Fold deu média 88% com desvio padrão alto, o modelo é bom?
- É melhor um modelo com 90% ± 10% ou 85% ± 2%?
- Se eu repetir o hold-out várias vezes e tirar a média, ainda preciso de K-Fold?
- Um modelo com alta acurácia pode ser ruim?

Referências

- GÉRON, AURÉLIEN. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. 2a Edição: O'Reilly Media, 2019.
    - **Capítulos 1 e 3**

C.E.S.A.R school

Diego Bezerra

dfb2@cesar.school

in/diegodefb